

로그캐치 Cypress

자동화 테스트 CI/CD 기술 문서

Auto-PC 기반 GitHub Actions 자동화 가이드

작성일	2026-06-09
담당자	윤호
버전	v1.0

목 차

1. 시스템 개요
 - 1.1 구성 요소
 - 1.2 전체 아키텍처
2. GitHub Actions 워크플로우 구조
 - 2.1 DEV_RELEASE_Check / DEV_ALPHA_Check
 - 2.2 Depth_fun_Check
 - 2.3 Basic_fun_Check / UI_Check
 - 2.4 Performance
 - 2.5 공통 알림 메커니즘
3. cypress.config.js 설명
 - 3.1 리포터 설정
 - 3.2 E2E 핵심 설정
 - 3.3 Node 이벤트 태스크
4. 신규 입사자 온보딩 가이드
 - 4.1 사전 준비 사항
 - 4.2 로컬 환경 설정
 - 4.3 GitHub Secrets 목록
 - 4.4 테스트 파일 구조
5. 개선 및 주의사항
 - 5.1 보안 이슈
 - 5.2 성능 개선
 - 5.3 코드 중복
 - 5.4 PR 자동 리뷰

1. 시스템 개요

본 시스템은 로그캐치 제품의 기능/UI/성능을 자동으로 검증하는 Cypress 기반 E2E 테스트 자동화 플랫폼입니다. GitHub Actions CI/CD와 연동되어 매일 정해진 시간에 자동으로 실행되며, 결과를 Google Chat 및 이메일로 즉시 통보합니다.

1.1 구성 요소

구성 요소	역할	버전/비고
Cypress	E2E 웹 자동화 테스트 프레임워크	최신 안정 버전
cy2	Cypress Cloud 병렬 실행 래퍼	병렬 스펙 분배
GitHub Actions	CI/CD 파이프라인 실행 환경	Self-hosted Runner (PC-Auto)
mochawesome	JSON/HTML 테스트 리포트 생성기	7.1.3
cypress-mochawesome-reporter	스크린샷 내장 리포터	cypress.config.js 연동
Lighthouse	웹 성능 측정 플러그인	@cypress-audit/lighthouse
node-ssh	SSH 원격 서버 접속 태스크	서버 검증용
oracledb / pg	Oracle/PostgreSQL DB 직접 조회	테스트 데이터 검증용

1.2 전체 아키텍처

워크플로우	트리거	실행 시간 (KST)
DEV_RELEASE_Check	Push(master) + Schedule	평일 02:05
DEV_ALPHA_Check	Push(master) + Schedule	평일 02:25
Depth_fun_Check	Push(master) + Schedule	평일 03:45
Basic_fun_Check	수동 (workflow_dispatch)	필요 시 수동
UI_Check	수동 (workflow_dispatch)	필요 시 수동
Performance	수동 (workflow_dispatch)	필요 시 수동

2. GitHub Actions 워크플로우 구조

2.1 DEV_RELEASE_Check / DEV_ALPHA_Check

배포 검증이 포함된 가장 복잡한 워크플로우입니다. Redmine 저장소에서 최신 빌드를 탐지하여 QA 서버에 자동 배포 후 테스트를 수행합니다.

Job	역할	비고
deploy-qa	Redmine에서 최신 버전 탐지 및 QA 서버에 SSH 배포	신규 버전 없으면 전체 SKIP
cypress-run	5개 병렬 러너로 E2E 테스트 실행	fail-fast: false
restart-task-run	순서 의존적인 태스크 파일 단독 실행	cypress-run 완료 후 실행
notify-result	JSON 병합 후 HTML 리포트 생성 + Google Chat/Email 발송	항상 실행 (if: always())

핵심 로직 deploy-qa의 outputs.skip_all 값으로 신규 버전 여부를 판단합니다. 동일 버전이면 cypress-run, restart-task-run, notify-result 모두 건너뛵니다.

2.2 Depth_fun_Check

심층 기능 검증 워크플로우로, 평일 새벽 3:45에 자동 실행됩니다. DB 접속(PostgreSQL, Oracle) 및 SSH 검증이 포함된 고급 시나리오를 실행합니다.

Job	역할	비고
cypress-run	1개 러너로 Depth 스펙 실행 (*11_Depth* 제외)	reporter 옵션 CLI 직접 주입
restart-task-run	*11_Depth* 파일 순차 단독 실행	패턴 배열로 확장 가능
notify-result	리포트 통합 및 Flaky 핀셋 추출 후 알림	실패 스펙 파일명 정확히 추출

2.3 Basic_fun_Check / UI_Check

기본 기능 및 UI 모듈 검증 워크플로우입니다. 현재 스케줄 없이 수동(workflow_dispatch)으로만 실행됩니다.

Job	역할	비고
cypress-run	5개 병렬 러너로 테스트 실행	*11_Basic* / UI 파일 제외 후 실행
restart-task-run	*11_Basic* 파일 단독 실행	Basic 기준

Job	역할	비고
notify-result	리포트 통합 + Flaky 방어 로직 + 알림	실패 시 이메일 첨부

2.4 Performance

Lighthouse 기반 웹 성능 측정 워크플로우입니다. 쾌적망(Fast)과 제한망(Limit) 두 단계로 나뉘어 순차 실행되며, 각 단계 완료 후 Google Chat으로 결과를 발송합니다.

Phase	대상 파일	네트워크 환경	비교 서버
Phase 1 (fast-run)	01, 02 파일	제한 없음 (사내망 속도)	10.10.54.41 vs 10.10.54.42
Phase 2 (limit-run)	03, 04 파일	네트워크 쓰로틀링 적용	10.10.54.41 vs 10.10.54.42

측정 방식 Lighthouse 리포트는 cypress/reports/lighthouse 폴더에 HTML로 저장됩니다. notify Job에서 가장 최신 파일을 기준으로 IP별 평균 점수를 계산하여 Google Chat으로 발송합니다.

2.5 공통 알림 메커니즘

모든 워크플로우의 notify-result Job은 동일한 3단계 Flaky 방어 로직으로 실패 여부를 판단합니다.

단계	검사 방법	목적
Step 1	개별 JSON 리포트에서 stats.failures > 0 확인	실제 테스트 실패 감지
Step 2	Job 결과 상태(failure, cancelled) 확인	시스템 크래시/취소 감지
Step 3	스크린샷 파일명에서 .cy.js 추출 (Step 1 실패 시만)	JSON 없는 극단적 실패 대비

3. cypress.config.js 설명

프로젝트 루트의 cypress.config.js는 테스트 실행 동작 전반을 제어하는 핵심 설정 파일입니다.

3.1 리포터 설정

옵션	값	설명
reporter	cypress-mochawesome-reporter	스크린샷 내장 가능한 Cypress 전용 리포터
reportDir	cypress/reports/json_logs	JSON 저장 경로 (YAML artifact 경로와 일치 필요)
reportFilename	report_\${random}	병렬 처리 시 파일명 충돌 방지용 랜덤 이름
html:false, json:true	—	HTML은 끄고 JSON만 저장 (marge로 나중에 합침)
embeddedScreenshots	true	실패 스크린샷을 JSON에 내장

주의 reportFilename에 Math.random() 대신 타임스탬프 사용을 권장합니다. 파일 정렬과 추적이 용이해집니다.

3.2 E2E 핵심 설정

옵션	값	설명
chromeWebSecurity	false	크로스 도메인 보안 정책 해제 (외부 WAS 접속용)
pageLoadTimeout	60000	페이지 로딩 대기 최대 60초 (WAS 응답 지연 대비)
defaultCommandTimeout	10000	기본 명령어 대기 10초
retries.runMode	2	CI 실행 시 실패한 테스트를 최대 2회 자동 재시도 (Flaky 방어)
viewportWidth/Height	1600 x 900	일관된 화면 크기 보장
numTestsKeptInMemory	0	메모리 절약 (대형 테스트 스위트 대비)
excludeSpecPattern	**/Check/**	Check 폴더 하위 파일은 기본 실행에서 제외

3.3 Node 이벤트 태스크 (setupNodeEvents)

cypress.config.js의 setupNodeEvents에는 테스트에서 cy.task()로 호출 가능한 서버사이드 기능들이 등록되어 있습니다.

태스크명	기능	사용 위치
lighthouse	Lighthouse 성능 리포트를 HTML 파일로 저장	Performance 테스트
setReportName	리포트 파일명 접두사 동적 변경	Lighthouse 파일명 구분
runSSH	SSH로 원격 서버 명령어 실행 후 결과 반환	서버 상태 검증
queryDB	Oracle DB 연결 및 SQL 쿼리 실행	DB 데이터 검증
queryPostgresDB	PostgreSQL 연결 및 쿼리 실행	LogCatch DB 검증
readDirectory	특정 폴더의 파일 목록 반환	다운로드 파일 확인
clearDownloads	다운로드 폴더 초기화	테스트 전 환경 정리
getFileStats	파일 크기 및 존재 여부 확인	파일 다운로드 검증

보안 주의 runSSH/queryDB/queryPostgresDB 태스크에 비밀번호 평문 fallback(|| "chakra" 등)이 포함되어 있습니다. 반드시 환경변수만 사용하도록 수정이 필요합니다.

4. 신규 입사자 온보딩 가이드

이 섹션은 테스트 자동화 환경에 처음 합류하는 팀원을 위한 설정 가이드입니다.

4.1 사전 준비 사항

필요 항목	설치/설정 방법
Node.js (v18 이상)	https://nodejs.org 에서 LTS 버전 설치
Git	https://git-scm.com 에서 설치 후 사용자 정보 설정
Chrome 브라우저	테스트 실행 기본 브라우저
Oracle Instant Client	C:\oracle\instantclient 경로에 설치 (Oracle DB 테스트 사용 시)
GitHub 저장소 접근 권한	담당자에게 Collaborator 권한 요청
GitHub Secrets 공유	담당자에게 아래 시크릿 값 확인 요청

4.2 로컬 환경 설정

1. 저장소 클론

```
git clone https://github.com/[organization]/[repository].git
cd [repository]
```

2. 패키지 설치

```
npm ci
```

3. .env 파일 생성 (프로젝트 루트, .gitignore 포함 — 절대 커밋 금지)

```
SSH_PASSWORD=
DB_PASSWORD=
PG_PASSWORD=
```

4. Cypress 바이너리 확인

```
npx cypress install
npx cypress verify
```

4.3 GitHub Secrets 목록

GitHub 저장소 Settings → Secrets and variables → Actions 에서 관리합니다.

Secret 이름	설명	사용 워크플로우
CYPRESS_RECORD_KEY	Cypress Cloud 대시보드 녹화 키	DEV_ALPHA/RELEASE
SSH_PRIVATE_KEY	QA 서버 SSH 접속용 개인키	DEV_ALPHA/RELEASE (deploy)
SSH_PASSWORD	SSH 접속 비밀번호	Depth (DB/SSH 테스트)

Secret 이름	설명	사용 워크플로우
DB_PASSWORD	Oracle DB 비밀번호	Depth
PG_PASSWORD	PostgreSQL 비밀번호	Depth
GOOGLE_CHAT_WEBHOOK	Google Chat 웹훅 URL	전체
MAIL_USERNAME	Gmail 발신 계정	전체 (실패 이메일)
MAIL_PASSWORD	Gmail 앱 비밀번호	전체 (실패 이메일)
REDMINE_USER_ID	Redmine 로그인 ID	DEV_ALPHA/RELEASE
REDMINE_PW	Redmine 로그인 비밀번호	DEV_ALPHA/RELEASE
QA_SERVER_USER	QA 서버 접속 계정명	DEV_RELEASE
QA_SERVER_IP	QA 서버 IP 주소	DEV_RELEASE

4.4 테스트 파일 구조

경로	내용
cypress/e2e/Basic_functions_check/	기본 기능 테스트 스펙 (Basic_fun_Check 워크플로우)
cypress/e2e/Depth_functions_check/	심층 기능 테스트 스펙 (Depth_fun_Check 워크플로우)
cypress/e2e/UI_Module_Full_Check/	UI 모듈 전체 검증 스펙 (UI_Check 워크플로우)
cypress/e2e/Basic_functions_Dev_check/	DEV 환경 기능 테스트 스펙
cypress/e2e/UI_Module_Dev_Check/	DEV 환경 UI 테스트 스펙
cypress/e2e/Performance/	성능 측정 스펙 (패적망)
cypress/e2e/Performance_Network_Limit/	성능 측정 스펙 (제한망)
cypress/reports/json_logs/	mochawesome JSON 리포트 저장 경로
cypress/reports/lighthouse/	Lighthouse HTML 리포트 저장 경로

5. 개선 및 주의사항

5.1 보안 이슈 (즉시 수정 권장)

긴급 아래 항목들은 코드가 GitHub에 공개되거나 다른 개발자에게 공유될 경우 보안 사고로 이어질 수 있습니다.

항목	현재 상태	권장 조치
cypress.config.js 비밀번호 fallback	'chakra', 'qwerty123' 등 평문 포함	fallback 제거 후 환경변수 미설정 시 에러 throw
Basic/Performance CYPRESS_RECORD_KEY	"my-secret-key" 하드코딩	secrets.CYPRESS_RECORD_KEY 사용
HARDCODED_VERSION	Basic, UI 워크플로우에 버전 수동 입력	배포 시 반드시 업데이트 또는 자동화

5.2 성능 개선 (npm 캐싱 추가 권장)

현재 모든 워크플로우에서 매 실행마다 npm ci와 npx cypress install을 반복합니다. 5개 병렬 러너 기준으로 실행당 수십 분이 낭비됩니다.

모든 워크플로우에 아래 캐싱 스텝 추가 권장:

```
- uses: actions/cache@v4
with:
path: |
node_modules
~/.cache/Cypress
key: ${{ runner.os }}-node-${{ hashFiles('package-lock.json') }}
```

5.3 코드 중복 (장기 개선)

중복 항목	중복 횟수	개선 방법
Google Chat 알림 로직 (~100줄)	5개 워크플로우	Reusable Workflow (_reusable-notify.yml) 분리
Create Dummy Report File 스텝	5개 워크플로우	Composite Action으로 추출
Extract Dashboard URL 스텝	4개 워크플로우	Composite Action으로 추출
deploy-qa Job	ALPHA/RELEASE 2개	input 파라미터로 분기 후 단일화

5.4 PR 자동 리뷰 (설정 완료)

PR이 열리거나 업데이트될 때 `.github/workflows/pr-auto-review.yml`이 자동으로 실행되어 아래 항목을 검사하고 PR 코멘트로 결과를 알려줍니다.

등급	검사 항목
수정 필요	CYPRESS_RECORD_KEY 하드코딩 · 민감정보 노출 · ci-build-id 불일치 · 비밀번호 평문 fallback · projectId 형식
개선 권장	HARCODED_VERSION · SSL 비활성화 · 캐싱 없음 · npm install 사용 · Math.random() 파일명 · cy.wait() 안티패턴
통과	retries 설정 확인 · viewport 설정 확인 · CYPRESS_RECORD_KEY 시크릿 사용

— 문서 끝 —